

# LabReSiD26 – Hands-On 14

Davide Ferrara – 518629

## 1 Esercizio 1: Conto corrente condiviso

$N = 4$  sportelli operano in parallelo sullo stesso conto corrente condiviso, eseguendo in loop operazioni di deposito e prelievo. Il prelievo è condizionato al saldo disponibile, che non deve mai scendere sotto zero.

### Race condition senza mutex

Il controllo `if (saldo >= importo)` da solo non è sufficiente. Se due thread leggono `saldo` contemporaneamente, entrambi trovano la condizione vera ed eseguono entrambi il prelievo: il saldo può diventare negativo e il risultato cambia ad ogni esecuzione. L'intera sequenza *leggi / controlla / modifica* deve essere atomica.

### Sezione critica con mutex

Si dichiara un `pthread_mutex_t` globale inizializzato staticamente con `PTHREAD_MUTEX_INITIALIZER`. Il mutex protegge sia il controllo `if` che la modifica di `saldo`: senza proteggere entrambi la race condition persiste.

```
1 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
2
3 /*
4  * Per N_OPERAZIONI si calcola un importo random che va
5  *   a 0 a 100.
6  * Se 'i' e' pari allora l'importo si somma altrimenti
7  *   si sottrae.
8  * Il controllo if (saldo >= importo) non e' sufficiente
9  *   in quanto la
10 * funzione sportello viene eseguita da piu thread e
11 *   saldo e' una
12 * variabile globale condivisa: se due thread scrivono
13 *   parallelamente
```

```

9  * uno potrebbe sovrascrivere il risultato dell'altro.
10 * Il MUTEX garantisce che solo un thread alla volta
    acceda alla
11 * sezione critica.
12 * */
13 void *sportello(void *arg) {
14     (void)arg;
15     unsigned int seed = 518629;
16     for (int i = 0; i < N_OPERAZIONI; i++) {
17         int importo = (rand_r(&seed) % 100) + 1;
18
19         // Sezione critica
20         pthread_mutex_lock(&mutex);
21         if (i % 2 == 0) {
22             saldo += importo;          /* deposito */
23         } else {
24             if (saldo >= importo)      /* prelievo */
25                 saldo -= importo;
26         }
27         pthread_mutex_unlock(&mutex);
28         // Fine sezione critica
29     }
30     return NULL;
31 }

```

Listing 1: Sezione critica protetta da mutex

## Riproducibilità con rand\_r

`rand()` usa uno stato interno globale non thread-safe: più thread che lo chiamano contemporaneamente generano una race condition sullo stato interno, producendo sequenze diverse ad ogni esecuzione. Si usa `rand_r(&seed)` con un seed locale per thread, dichiarato fuori dal loop così che lo stato si aggiorni tra un'iterazione e l'altra.

```

1 $ ./banca
2 Saldo finale: 131056 euro
3 $ ./banca
4 Saldo finale: 131056 euro
5 $ ./banca
6 Saldo finale: 131056 euro

```

Listing 2: Test Esercizio 1 – risultato riproducibile

## 2 Esercizio 2: Stampante condivisa con timeout

Un ufficio ha una sola stampante condivisa tra  $N = 6$  utenti. Ogni utente tenta di acquisire la stampante: se non si libera entro 3 secondi rinuncia. Una stampa richiede 2 secondi e gli utenti arrivano uno ogni 0.2 secondi.

### Timeout con `pthread_mutex_timedlock`

`pthread_mutex_timedlock()` richiede un tempo **assoluto** basato su `CLOCK_REALTIME`. Si legge prima l'ora corrente con `clock_gettime()` e poi si aggiungono i secondi di attesa. Le funzioni `pthread_*` restituiscono il codice di errore direttamente come valore di ritorno, senza settare `errno`. Se restituisce `ETIMEDOUT` il mutex **non** è stato acquisito e chiamare `unlock()` in questo caso produrrebbe comportamento indefinito.

```
1 void *utente(void *arg) {
2     int id = *(int *)arg;
3
4     /* Ora di sistema attuale (CLOCK_REALTIME) + 3 secondi */
5     struct timespec ts;
6     clock_gettime(CLOCK_REALTIME, &ts);
7     ts.tv_sec += 3;
8
9     /* Provo ad acquisire il mutex per 3 secondi */
10    int rc = pthread_mutex_timedlock(&stampante, &ts);
11    if (rc == 0) {
12        printf("[Utente %d] Sta stampando... \n", id);
13        sleep(2); /* Simula la stampa */
14        printf("[Utente %d] Ha terminato la stampa :D \n",
15              id);
16        pthread_mutex_unlock(&stampante);
17    } else if (rc == ETIMEDOUT) {
18        printf("[Utente %d] Rinuncia alla stampa :(\n", id);
19    } else {
20        printf("[Utente %d] Errore nell'acquisizione del mutex
21              .\n", id);
22    }
23    return NULL;
24 }
```

Listing 3: Funzione utente con timeout

## Analisi del comportamento

Gli utenti arrivano ogni 0.2s, ogni stampa dura 2s e il timeout è 3s. La timeline permette di ragionare su chi riesce a stampare:

```
1 /*
2  * 0, 0.2, 0.4, 0.6, 0.8, 1, 1.2, 1.4, 1.6, 1.8, 2,
3  *   2.2, ...
4  * u0(stampa) -> sleep(2) ..... u0(termina)
5  *   u1(attende) -> wait(3) ..... u1(stampa)
6  *   -> ...
7  *   u2(attende) -> wait(3)
8  *   ..... u2(rinuncia)
9  * */
```

```
1 $ ./stampante
2 [Utente 0] Sta stampando...
3 [Utente 0] Ha terminato la stampa :D
4 [Utente 1] Sta stampando...
5 [Utente 2] Rinuncia alla stampa :(
6 [Utente 3] Rinuncia alla stampa :(
7 [Utente 4] Rinuncia alla stampa :(
8 [Utente 1] Ha terminato la stampa :D
9 [Utente 5] Sta stampando...
10 [Utente 5] Ha terminato la stampa :D
```

Listing 4: Test Esercizio 2 – stampa 2s timeout 3s

## 3 Esercizio 3: Rubrica telefonica con rwlock

Una rubrica è condivisa tra  $N = 8$  thread lettori e un thread scrittore. I lettori fanno ricerche frequenti e possono leggere in parallelo tra loro; lo scrittore aggiorna i contatti raramente ma richiede accesso esclusivo. Un mutex ordinario serializzerebbe anche le letture tra loro, inutilmente. `pthread_rwlock_t` risolve questo: più lettori possono tenere il lock simultaneamente, mentre lo scrittore ottiene accesso esclusivo.

### Lettori in parallelo

Ogni lettore acquisisce il lock in lettura con `pthread_rwlock_rdlock()`, legge il contatto e rilascia. Più lettori possono tenere il lock contemporaneamente, perché leggere in parallelo è sempre safe.

```

1 void *lettore(void *arg) {
2     int id = *(int *)arg;
3     for (int i = 0; i < 5; i++) {
4         // Acquisisce il lock in lettura
5         pthread_rwlock_rdlock(&rubrica_lock);
6
7         int idx = i % n_contatti;
8         printf("Lettore %d: %s -> %s\n", id, rubrica[idx].
9             nome,
10                rubrica[idx].numero);
11         usleep(100000);
12
13         // Rilascia il lock
14         pthread_rwlock_unlock(&rubrica_lock);
15     }
16     return NULL;
17 }

```

Listing 5: Thread lettore

## Scrittore con accesso esclusivo

Lo scrittore dorme 1 secondo per lasciare partire i lettori, poi acquisisce il lock in scrittura con `pthread_rwlock_wrlock()`: blocca finché tutti i lettori attivi non rilasciano il lock, poi ottiene accesso esclusivo (scrive solamente quando nessuno legge).

```

1 void *scrittore(void *arg) {
2     (void)arg;
3     sleep(1); /* lascia partire i lettori */
4
5     pthread_rwlock_wrlock(&rubrica_lock);
6
7     printf("Scrittore: aggiornamento il numero di Bob...\n");
8     strcpy(rubrica[1].numero, "333-9999");
9     sleep(2); /* simula aggiornamento lento */
10    printf("Scrittore: aggiornamento completato.\n");
11
12    pthread_rwlock_unlock(&rubrica_lock);
13    return NULL;
14 }

```

Listing 6: Thread scrittore

```

1 $ ./rubrica
2 Lettore 0: Alice -> 333-1111          # prima tornata:
   lettori in parallelo
3 Lettore 3: Alice -> 333-1111          # (interleaving
   visibile negli id)
4 ...
5 Lettore 0: Bob -> 333-2222
6 Lettore 4: Bob -> 333-2222
7 Lettore 2: Bob -> 333-2222
8 ...
9 Scrittore: aggiorno il numero di Bob...
10                                     # blocco
                                     silenzioso 2s
                                     (wrlock
                                     esclusivo)
11 Scrittore: aggiornamento completato.
12 Lettore 0: Alice -> 333-1111          # seconda tornata:
   lettori ripartono
13 Lettore 1: Alice -> 333-1111
14 ...
15 Lettore 0: Bob -> 333-9999            # numero
   aggiornato visibile a tutti
16 Lettore 3: Bob -> 333-9999
17 Lettore 1: Bob -> 333-9999
18 ...

```

Listing 7: Test Esercizio 3 – interleaving, blocco scrittore e aggiornamento

L’interleaving degli id nei round di lettura conferma il parallelismo dei lettori. Il blocco silenzioso di 2 secondi tra i messaggi dello scrittore dimostra l’accesso esclusivo del wrlock. Nella seconda tornata tutti i lettori leggono 333-9999: nessuno vede il vecchio valore dopo l’aggiornamento. Inoltre il programma non segnala nessuna race condition che sarebbe comparsa compilando con `-fsanitize=thread`.